Blocks

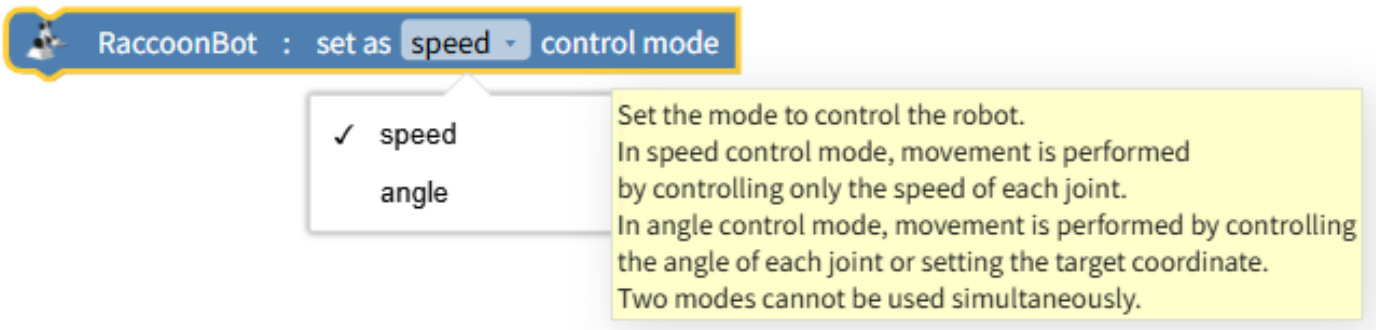
Set Control Mode

Sets the mode for controlling the RaccoonBot.

In speed control mode, joints are moved only through the speed of each joint.

In angle control mode, control is performed using the angle control speed and each joint angle, or by moving through a target coordinate.

The two modes cannot be used simultaneously.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
mode	Dropdown option	RaccoonBot control mode	speed (0), angle (1)

JavaScript Code

```
// Set the RaccoonBot to speed control mode
$('Raccoon4*0:mode').d = 0;

// Set the RaccoonBot to angle control mode
$('Raccoon4*0:mode').d = 1;
$('Raccoon4*0:angle.joints').d = [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d];
```

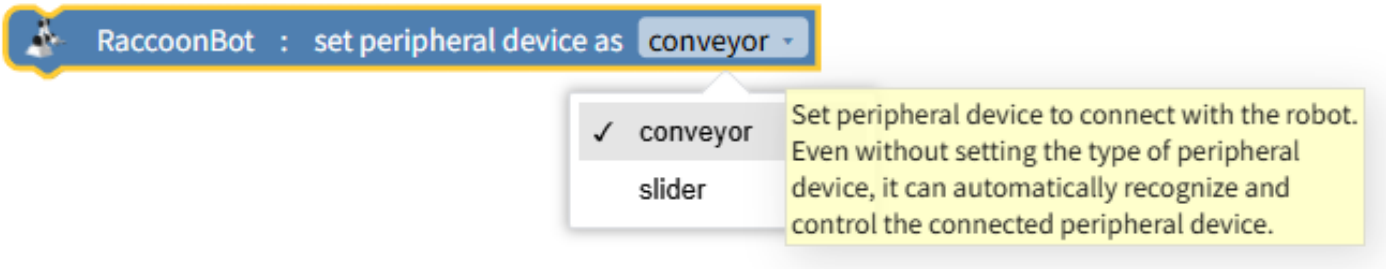
Python Code

```python
# Set the RaccoonBot to speed control mode
__('Raccoon4*0:mode').d = 0

# Set the RaccoonBot to angle control mode
__('Raccoon4*0:mode').d = 1
__('Raccoon4*0:angle.joints').d = [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d]
```

Set Peripheral Devices

Sets peripheral devices to be used connected with the RaccoonBot.

Even if you do not set it separately, it can automatically recognize and control the connected peripheral devices.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
peripheral	Dropdown option	Peripheral device	conveyor (1), slider (2)

JavaScript Code

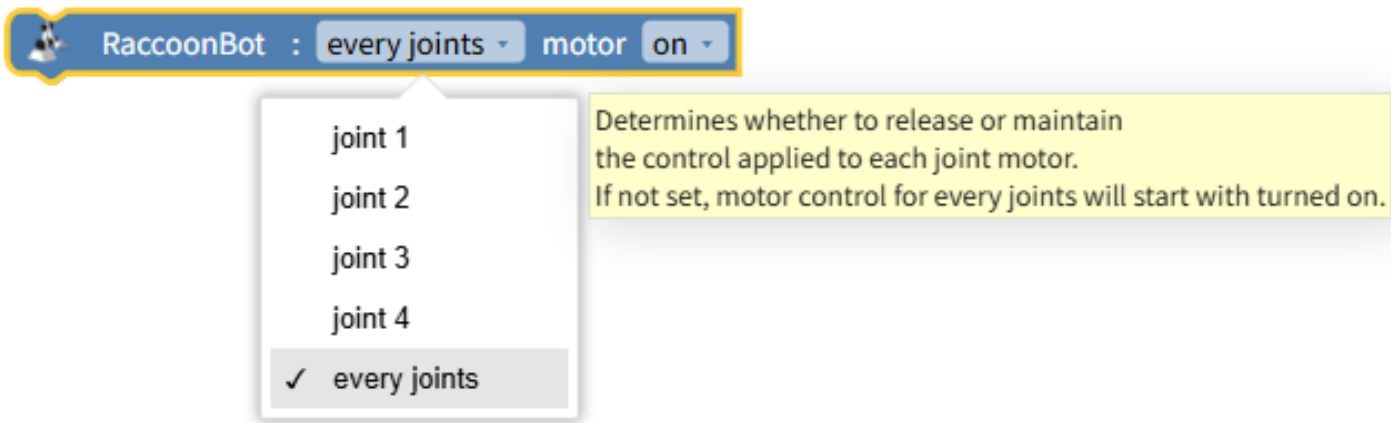
```javascript
// Set the peripheral device to Conveyor
$('Raccoon4*0:peripheral>').d = 1;

// Set the peripheral device to Slider
$('Raccoon4*0:peripheral>').d = 2;
```

Python Code

```
# Set the peripheral device to Conveyor
__('Raccoon4*0:peripheral>').d = 1

# Set the peripheral device to Slider
__('Raccoon4*0:peripheral>').d = 2
```

Turn Joint Motor Control On / Off

Determines whether to release or maintain the control applied to each joint motor.
If it is not set, it starts with motor control turned on for all joints.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
motor	Dropdown option	Joint number	joint 1, joint 2, joint 3, joint 4, every joints
power	Dropdown option	On/Off	on (0), off (1)

JavaScript Code

```
// Turn on motor control for joint 1
$('Raccoon4*0:motor_off.joint_1').d = 0;  // on

// Turn off motor control for joint 2
```

```
$('Raccoon4*0:motor_off.joint_2').d = 1;  // off

// Turn on motor control for joint 3
$('Raccoon4*0:motor_off.joint_3').d = 0;  // on

// Turn off motor control for joint 4
$('Raccoon4*0:motor_off.joint_4').d = 1;  // off

// Turn on motor control for every joints
$('Raccoon4*0:motor_off.joint_1').d = 0;  // on
$('Raccoon4*0:motor_off.joint_2').d = 0;  // on
$('Raccoon4*0:motor_off.joint_3').d = 0;  // on
$('Raccoon4*0:motor_off.joint_4').d = 0;  // on
```

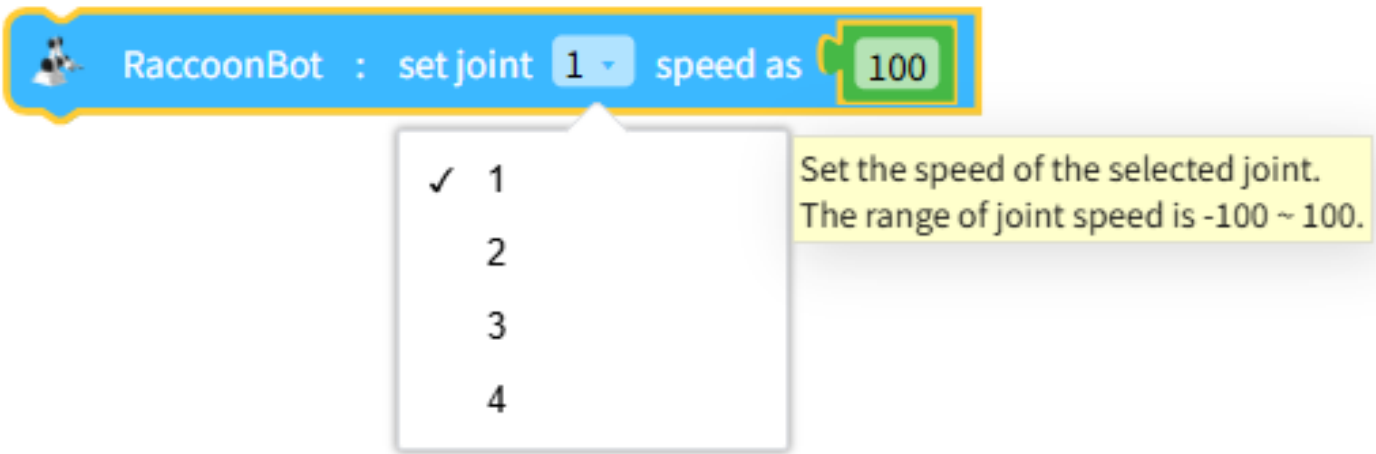
Python Code

```
# Turn on motor control for joint 1
__('Raccoon4*0:motor_off.joint_1').d = 0  # on

# Turn off motor control for joint 2
__('Raccoon4*0:motor_off.joint_2').d = 1  # off

# Turn on motor control for joint 3
__('Raccoon4*0:motor_off.joint_3').d = 0  # on

# Turn off motor control for joint 4
__('Raccoon4*0:motor_off.joint_4').d = 1  # off

# Turn on motor control for every joints
__('Raccoon4*0:motor_off.joint_1').d = 0  # on
__('Raccoon4*0:motor_off.joint_2').d = 0  # on
__('Raccoon4*0:motor_off.joint_3').d = 0  # on
__('Raccoon4*0:motor_off.joint_4').d = 0  # on
```

Set Joint Speed

Sets the selected joint speed.

The joint speed range is -100 ~ 100.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
motor	Dropdown option	Joint number	1, 2, 3, 4
speed	Input value	Joint speed	Real number -100 ~ 100

JavaScript Code

```
// Set joint 1 speed to 100
$('Raccoon4*0:speed.joint_1').d = 100;

// Set joint 2 speed to 90
$('Raccoon4*0:speed.joint_2').d = 90;

// Set joint 3 speed to 80
$('Raccoon4*0:speed.joint_3').d = 80;

// Set joint 4 speed to 70
$('Raccoon4*0:speed.joint_4').d = 70;
```

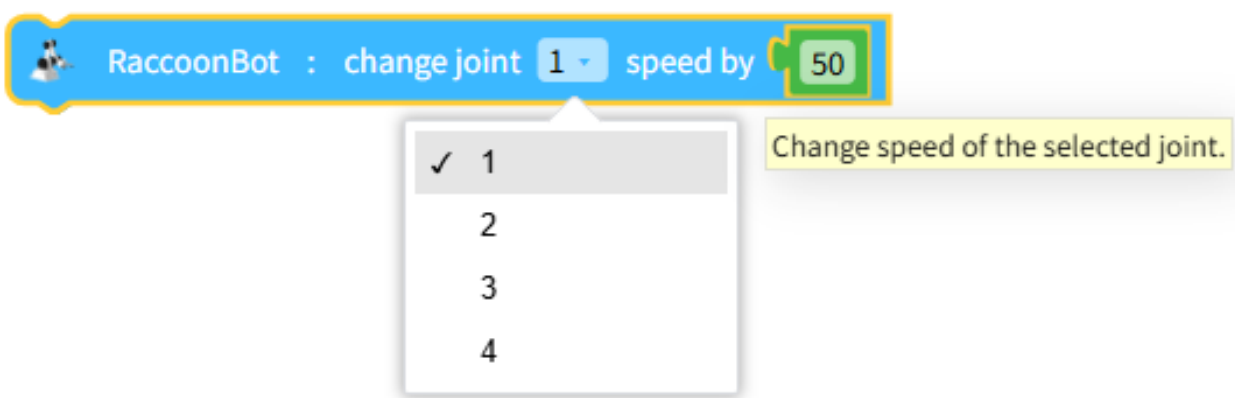
Python Code

```
# Set joint 1 speed to 100
__('Raccoon4*0:speed.joint_1').d = 100

# Set joint 2 speed to 90
__('Raccoon4*0:speed.joint_2').d = 90

# Set joint 3 speed to 80
__('Raccoon4*0:speed.joint_3').d = 80

# Set joint 4 speed to 70
__('Raccoon4*0:speed.joint_4').d = 70
```

Change Joint Speed

Changes the selected joint speed by the input value.

The input value range is -200 ~ 200.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
motor	Dropdown option	Joint number	1, 2, 3, 4
speed	Input value	Speed change	Real number -200 ~ 200

JavaScript Code

```
// Change joint 1 speed by -200
$('Raccoon4*0:speed.joint_1').d = $('Raccoon4*0:speed.joint_1').d + (-200);

// Change joint 2 speed by 40
$('Raccoon4*0:speed.joint_2').d = $('Raccoon4*0:speed.joint_2').d + 40;

// Change joint 3 speed by 30
$('Raccoon4*0:speed.joint_3').d = $('Raccoon4*0:speed.joint_3').d + 30;

// Change joint 4 speed by 20
$('Raccoon4*0:speed.joint_4').d = $('Raccoon4*0:speed.joint_4').d + 20;
```

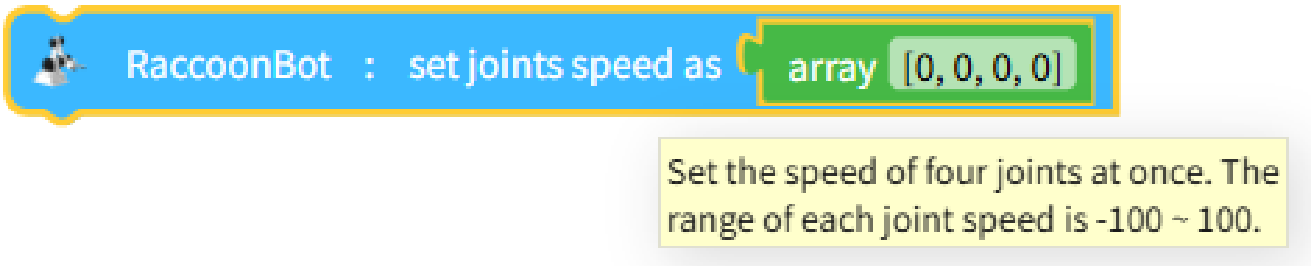
Python Code

```
# Change joint 1 speed by -200
__('Raccoon4*0:speed.joint_1').d = __('Raccoon4*0:speed.joint_1').d + (-200)

# Change joint 2 speed by 40
__('Raccoon4*0:speed.joint_2').d = __('Raccoon4*0:speed.joint_2').d + 40

# Change joint 3 speed by 30
__('Raccoon4*0:speed.joint_3').d = __('Raccoon4*0:speed.joint_3').d + 30

# Change joint 4 speed by 20
__('Raccoon4*0:speed.joint_4').d = __('Raccoon4*0:speed.joint_4').d + 20
```

Set Joint Speeds as an Array

Sets the speeds of the four joints at once.

The speed range for each joint is -100 ~ 100.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
speed	Input value	Speed	Array of real numbers between -100 and 100

JavaScript Code

```
// Set each joint speed to [10, 20, 30, 40]
$('Raccoon4*0:speed.joint_1').d = 10;
$('Raccoon4*0:speed.joint_2').d = 20;
$('Raccoon4*0:speed.joint_3').d = 30;
$('Raccoon4*0:speed.joint_4').d = 40;
```

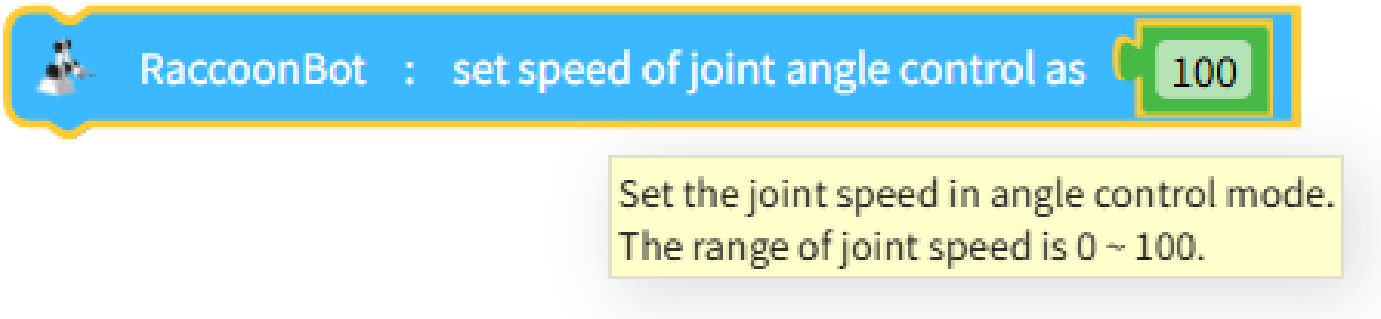
Python Code

```
# Set each joint speed to [10, 20, 30, 40]
__('Raccoon4*0:speed.joint_1').d = 10
__('Raccoon4*0:speed.joint_2').d = 20
__('Raccoon4*0:speed.joint_3').d = 30
__('Raccoon4*0:speed.joint_4').d = 40
```

Set Joint Angle Control Speed

Sets the speed to control joints in angle control mode.

The speed range is 0 ~ 100.



Dropdown Options and Input Values

Name | Type | Description | Range / Type |

speed | Input value | Joint control speed | Real number 0 ~ 100 |

JavaScript Code

```
// Set the joint angle control speed to 100
$('Raccoon4*0:angle.max_speed').d = 100;
```

Python Code

```
# Set the joint angle control speed to 100
__('Raccoon4*0:angle.max_speed').d = 100
```

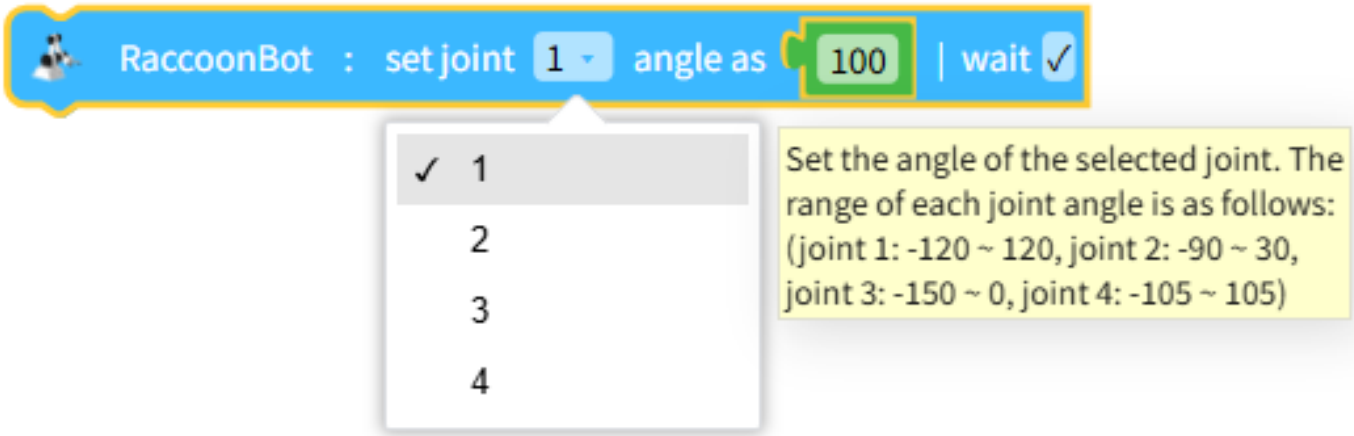
Set Joint Angle

Sets the angle of the selected joint. The angle range for each joint is as follows:

Joint 1: -120 ~ 120, Joint 2: -90 ~ 30, Joint 3: -150 ~ 0, Joint 4: -105 ~ 105

If you check Wait, it waits until the movement is completed.

However, if Wait is checked, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
motor	Dropdown option	Joint number	1, 2, 3, 4
angle	Input value	Joint angle	Joint 1: -120 ~ 120

Joint 2: -90 ~ 30 Joint 3: -150 ~ 0 Joint 4: -105 ~ 105|

JavaScript Code

```
// Set joint 1 angle to 100 | Wait O
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 1) ? 100 : Math.floor(data));
await $('Raccoon4*0:angle.!joints').w();

// Set joint 2 angle to 30 | Wait O
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 2) ? 30 : Math.floor(data));
await $('Raccoon4*0:angle.!joints').w();

// Set joint 3 angle to 0 | Wait X
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 3) ? 0 : Math.floor(data));
```

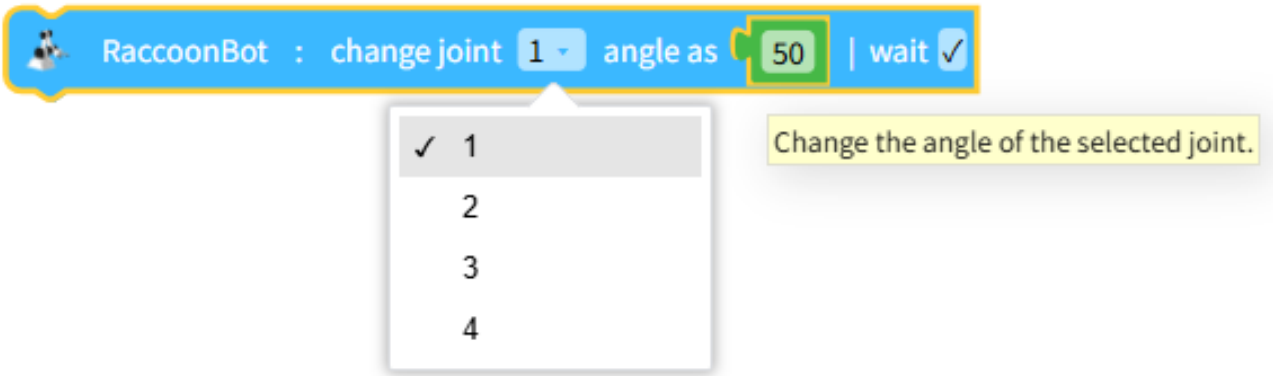
```
// Set joint 4 angle to 100 | Wait X
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 4) ? 100 : Math.floor(data));
```

Python Code

```
# Set joint 1 angle to 100 | Wait O
__('Raccoon4*0:angle.joints').d = [100 if i + 1 == 1 else
__('Raccoon4*0:angle.joints').d[i] for i in range(4)]
await __('Raccoon4*0:angle.!joints').w()

# Set joint 2 angle to 30 | Wait O
__('Raccoon4*0:angle.joints').d = [30 if i + 1 == 2 else
__('Raccoon4*0:angle.joints').d[i] for i in range(4)]
await __('Raccoon4*0:angle.!joints').w()

# Set joint 3 angle to 0 | Wait X
__('Raccoon4*0:angle.joints').d = [0 if i + 1 == 3 else
__('Raccoon4*0:angle.joints').d[i] for i in range(4)]

# Set joint 4 angle to 100 | Wait X
__('Raccoon4*0:angle.joints').d = [100 if i + 1 == 4 else
__('Raccoon4*0:angle.joints').d[i] for i in range(4)]
```

Change Joint Angle

Changes the angle of the selected joint by the input value. The range of each input value is as follows:
Joint 1: -240 ~ 240, Joint 2: -120 ~ 120, Joint 3: -150 ~ 150, Joint 4: -210 ~ 210
If you check Wait, it waits until the movement is completed.
However, if Wait is checked, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
motor	Dropdown option	Joint number	1, 2, 3, 4
angle	Input value	Joint change value	Joint 1: -240 ~ 240

Joint 2: -120 ~ 120 Joint 3: -150 ~ 150 Joint 4: -210 ~ 210|

JavaScript Code

```
// Change joint 1 angle by 50 | Wait O
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 1) ? data + 50 : data);
await $('Raccoon4*0:angle.!joints').w();

// Change joint 2 angle by 40 | Wait O
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 2) ? data + 40  : data);
await $('Raccoon4*0:angle.!joints').w();

// Change joint 3 angle by 30 | Wait X
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 3) ? data + 30 : data);

// Change joint 4 angle by 20 | Wait X
$('Raccoon4*0:angle.joints').d = $('Raccoon4*0:angle.joints').d.map((data, i) =>
(i + 1 === 4) ? data + 20 : data);
```

Python Code

```
# Change joint 1 angle by 50 | Wait O
__('Raccoon4*0:angle.joints').d = [__('Raccoon4*0:angle.joints').d[i] + 50 if i +
1 == 1 else __('Raccoon4*0:angle.joints').d[i] for i in range(4)]
await __('Raccoon4*0:angle.!joints').w()

# Change joint 2 angle by 40 | Wait O
__('Raccoon4*0:angle.joints').d = [__('Raccoon4*0:angle.joints').d[i] + 40 if i +
1 == 2 else __('Raccoon4*0:angle.joints').d[i] for i in range(4)]
await __('Raccoon4*0:angle.!joints').w()

# Change joint 3 angle by 30 | Wait X
__('Raccoon4*0:angle.joints').d = [__('Raccoon4*0:angle.joints').d[i] + 30 if i +
1 == 3 else __('Raccoon4*0:angle.joints').d[i] for i in range(4)]

# Change joint 4 angle by 20 | Wait X
__('Raccoon4*0:angle.joints').d = [__('Raccoon4*0:angle.joints').d[i] + 20 if i +
1 == 4 else __('Raccoon4*0:angle.joints').d[i] for i in range(4)]
```

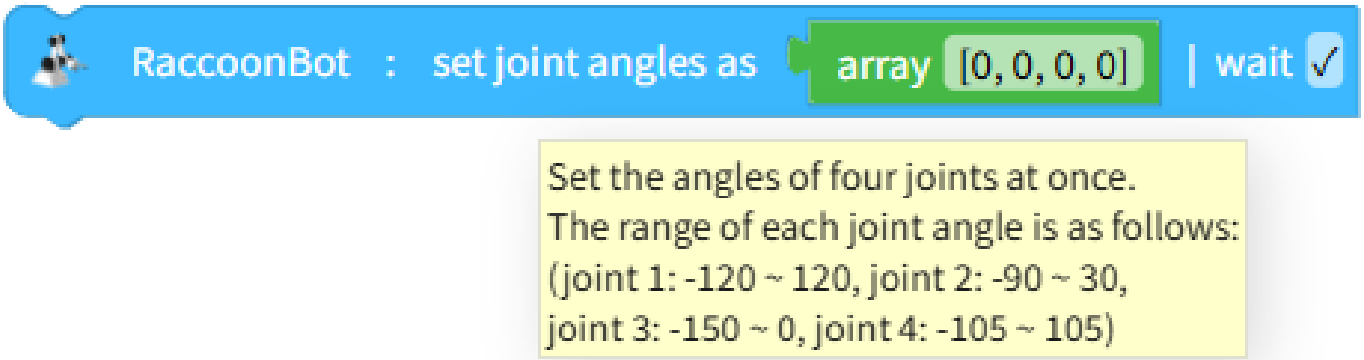
Set Joint Angles as an Array

Sets the angles of the four joints at once. The angle range for each joint is as follows:

Joint 1: -120 ~ 120, Joint 2: -90 ~ 30, Joint 3: -150 ~ 0, Joint 4: -105 ~ 105

If you check Wait, it waits until the movement is completed.

However, if Wait is checked, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
angle	Input value	Angle	Joint 1: -120 ~ 120

Joint 2: -90 ~ 30 Joint 3: -150 ~ 0 Joint 4: -105 ~ 105|

JavaScript Code

```
// Set each joint angle to [0, 0, 0, 0] | Wait X
$('Raccoon4*0:angle.joints').d = [0,0,0,0];

// Set each joint angle to [10, 10, -10, 10] | Wait O
$('Raccoon4*0:angle.joints').d = [10,10,-10,10];
await $('Raccoon4*0:angle.!joints').w();
```

Python Code

```
# Set each joint angle to [0, 0, 0, 0] | Wait X
__('Raccoon4*0:angle.joints').d = [0,0,0,0]

# Set each joint angle to [10, 10, -10, 10] | Wait O
__('Raccoon4*0:angle.joints').d = [10,10,-10,10]
await __('Raccoon4*0:angle.!joints').w()
```

Move by XYZ Coordinates

Sets x, y, z coordinates to move the robot arm.

The range of each coordinate is as follows:

wrist => x: -200mm ~ 200mm, y: -100mm ~ 200mm, z: -20mm ~ 280mm

end-effector => x: -280mm ~ 280mm, y: -180mm ~ 280mm, z: -100mm ~ 360mm

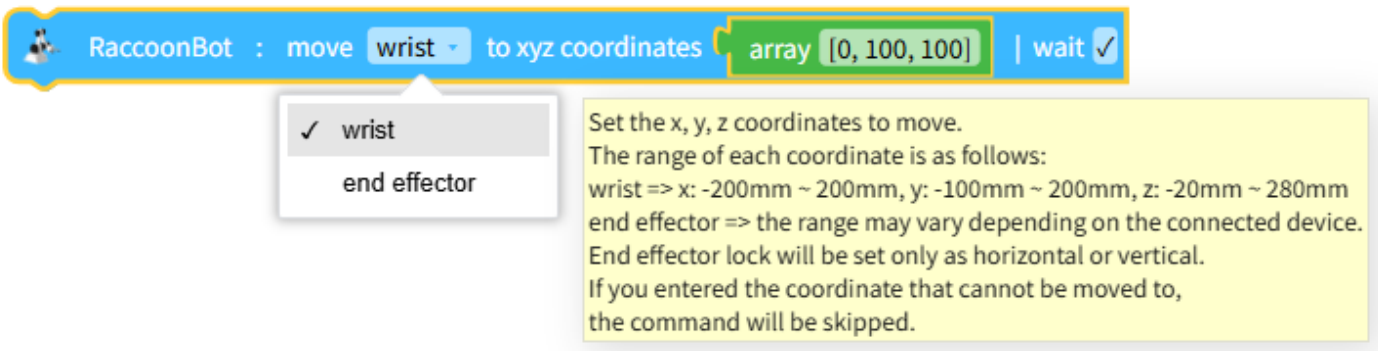
When controlling based on the end-effector, the range may vary depending on the connected device.

The direction of the end-effector is fixed horizontally or vertically to the ground.

If you enter coordinates that cannot be reached, the command is skipped.

If you check Wait, it waits until the movement is completed.

However, if Wait is checked, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
origin	Dropdown option	Coordinate reference	wrist (wrist), end-effector (end_effector)
coordinate	Input value	Target coordinate x,y,z	Real number array

JavaScript Code

```
// Move to [0, 100, 100] x,y,z based on end-effector | Wait O
$('Raccoon4*0:angle.joints').d = __xyz_to_angles('Raccoon4*0', [0,100,100],
'end_effector');
await $('Raccoon4*0:angle.!joints').w();
```

```
// Move to [0, 100, 100] x,y,z based on wrist | Wait X
$('Raccoon4*0:angle.joints').d = __xyz_to_angles('Raccoon4*0', [0,100,100],
'wrist');
```

Python Code

```
# Move to [0, 100, 100] x,y,z based on end-effector | Wait O
__('Raccoon4*0:angle.joints').d = __xyz_to_angles('Raccoon4*0', [0,100,100],
'end_effector')
await __('Raccoon4*0:angle.!joints').w()

# Move to [0, 100, 100] x,y,z based on wrist | Wait X
__('Raccoon4*0:angle.joints').d = __xyz_to_angles('Raccoon4*0', [0,100,100],
'wrist')
```

Initialize Joint State

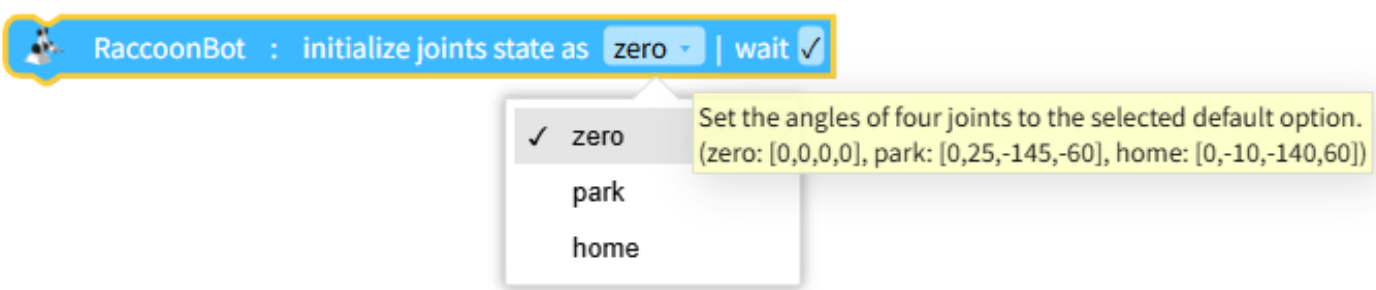
Sets the angles of the four joints to the selected default option.

The angles of the default options are as follows:

zero: [0,0,0,0], park: [0,25,-145,-60], home: [0,-10,-140,60]

If you check Wait, it waits until the movement is completed.

However, if Wait is checked, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
option	Dropdown option	Default option	zero, park, home

JavaScript Code

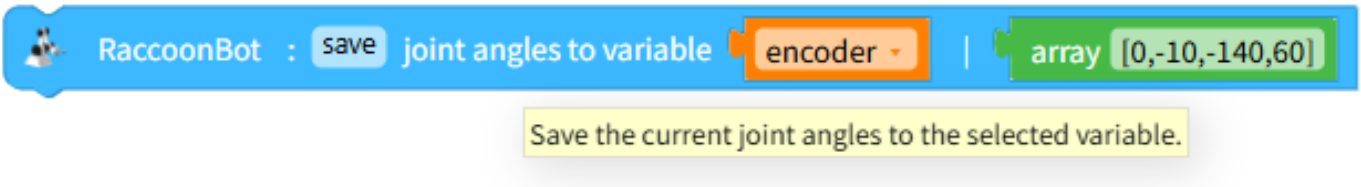
```
// Initialize joint state to zero | Wait O
$('Raccoon4*0:angle.joints').d = [0, 0, 0, 0];
await $('Raccoon4*0:angle.!joints').w();

// Initialize joint state to park | Wait X
$('Raccoon4*0:angle.joints').d = [0, 25, -145, -60];

// Initialize joint state to home | Wait X
$('Raccoon4*0:angle.joints').d = [0, -10, -140, 60];
```

Python Code

```
# Initialize joint state to zero | Wait O
__('Raccoon4*0:angle.joints').d = [0, 0, 0, 0]
await __('Raccoon4*0:angle.!joints').w()

# Initialize joint state to park | Wait X
__('Raccoon4*0:angle.joints').d = [0, 25, -145, -60]

# Initialize joint state to home | Wait X
__('Raccoon4*0:angle.joints').d = [0, -10, -140, 60]
```

Save Joint Angles

When you press the Save button, it saves the current joint angle values into the specified variable. Also, it receives the current joint angle values as an array at the same time.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
variable	Input value	Variable to receive joint values	Block composer variable

JavaScript Code

```
// When the current angle value is [0,-10,-140,60], upon clicking Save
var encoder;
encoder = [0,-10,-140,60];
```

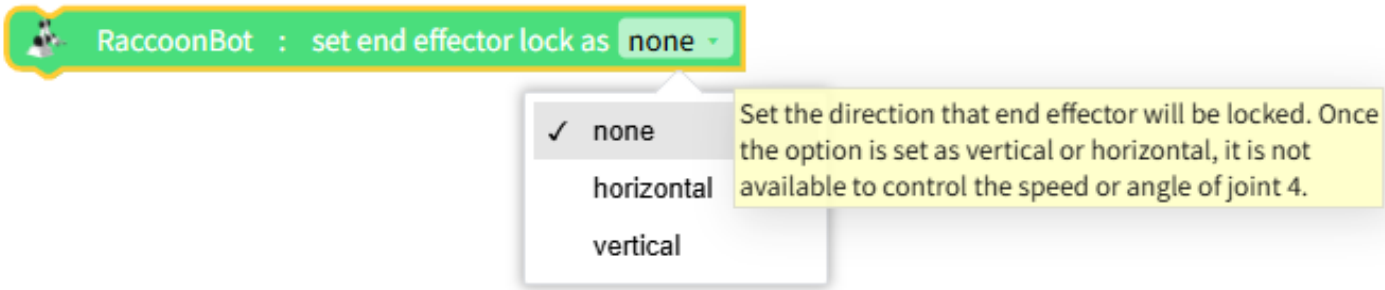
Python Code

```
# When the current angle value is [0,-10,-140,60], upon clicking Save
encoder = None
encoder = [0,-10,-140,60]
```

Set End-Effector Lock

Sets the direction to lock the end-effector.

After the option is set to horizontal or vertical, you cannot control the speed or angle of joint 4.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
fix	Dropdown option	Lock setting value	None (0), Horizontal (1), Vertical (2)

JavaScript Code

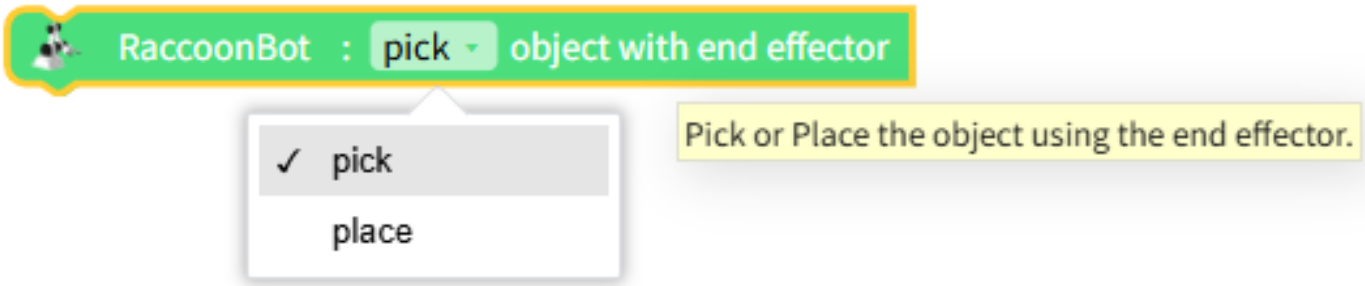
```
// Set end-effector lock to horizontal
$('Raccoon4*0:end_effector.lock').d = 1;

// Set end-effector lock to vertical
$('Raccoon4*0:end_effector.lock').d = 2;

// Set end-effector lock to none
$('Raccoon4*0:end_effector.lock').d = 0;
```

Python Code

```
# Set end-effector lock to horizontal
__('Raccoon4*0:end_effector.lock').d = 1

# Set end-effector lock to vertical
__('Raccoon4*0:end_effector.lock').d = 2

# Set end-effector lock to none
__('Raccoon4*0:end_effector.lock').d = 0
```

Pick / Place an Object with the End-Effector

Uses the end-effector to pick or place an object.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
toggle	Dropdown option	Whether to use end-effector	pick (1), place (0)

JavaScript Code

```
// Pick an object with the end-effector
$('Raccoon4*0:end_effector.control').d = 1;

// Place an object with the end-effector
$('Raccoon4*0:end_effector.control').d = 0;
```

Python Code

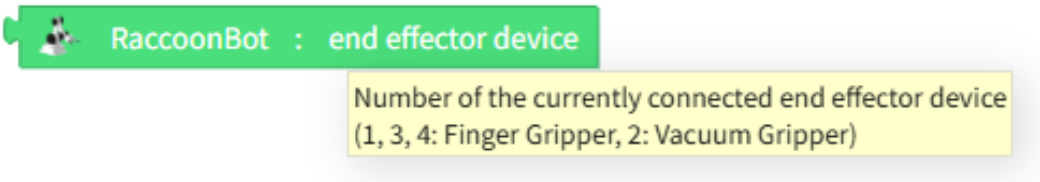
```
# Pick an object with the end-effector
__('Raccoon4*0:end_effector.control').d = 1

# Place an object with the end-effector
__('Raccoon4*0:end_effector.control').d = 0
```

End-Effector Value

Returns a number depending on the information of the currently connected end-effector device:

1, 3, 4 -> finger gripper

2 -> vacuum gripper



JavaScript Code

```
$('Raccoon4*0:end_effector.device').d; // end-effector device info value
```

Python Code

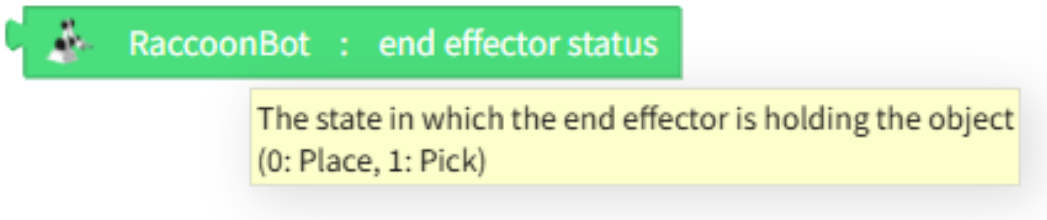
```
__('Raccoon4*0:end_effector.device').d # end-effector device info value
```

End-Effector Status

Returns a numeric value based on whether the end-effector is holding an object:

0 -> Place state

1 -> Pick state



JavaScript Code

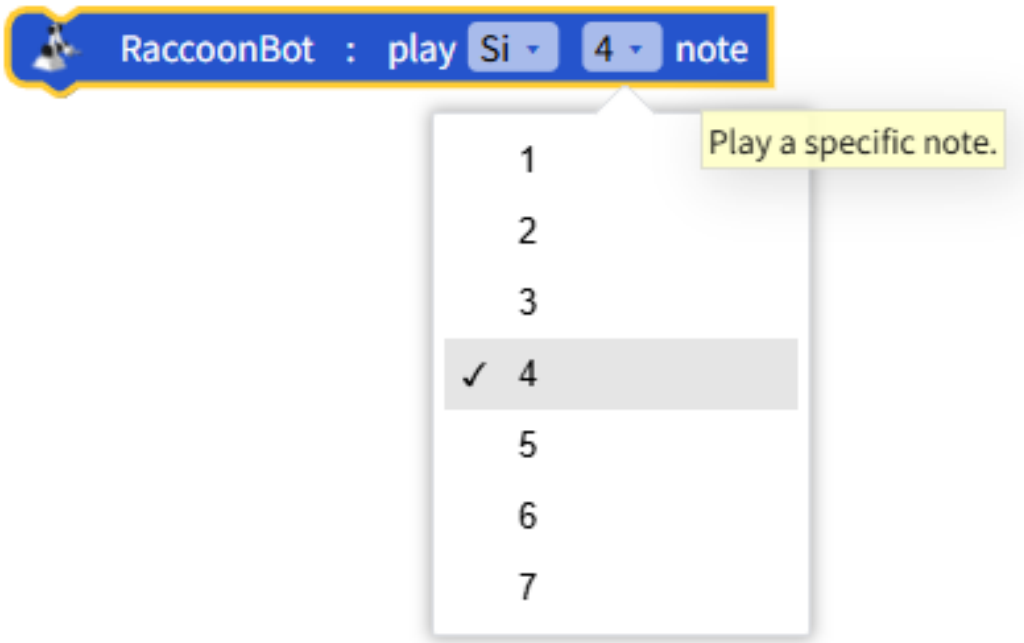
```
$('Raccoon4*0:end_effector.status').d; // end-effector status value
```

Python Code

```
__('Raccoon4*0:end_effector.status').d # end-effector status value
```

Play a Scale

The RaccoonBot plays the specified scale.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
note	Dropdown option	Note	Do, Do#, Re, Re#, Mi, Fa, Fa#, Sol, Sol#, La, La#, Si
octave	Dropdown option	Octave	1 ~ 7

JavaScript Code

```
// Play Do in octave 1
$('Raccoon4*0:sound.note').d = 4;

// Play Re in octave 1
$('Raccoon4*0:sound.note').d = 6;

// Play Do in octave 2
```

```
$('Raccoon4*0:sound.note').d = 16;

// Play Si in octave 7
$('Raccoon4*0:sound.note').d = 87;
```

Python Code

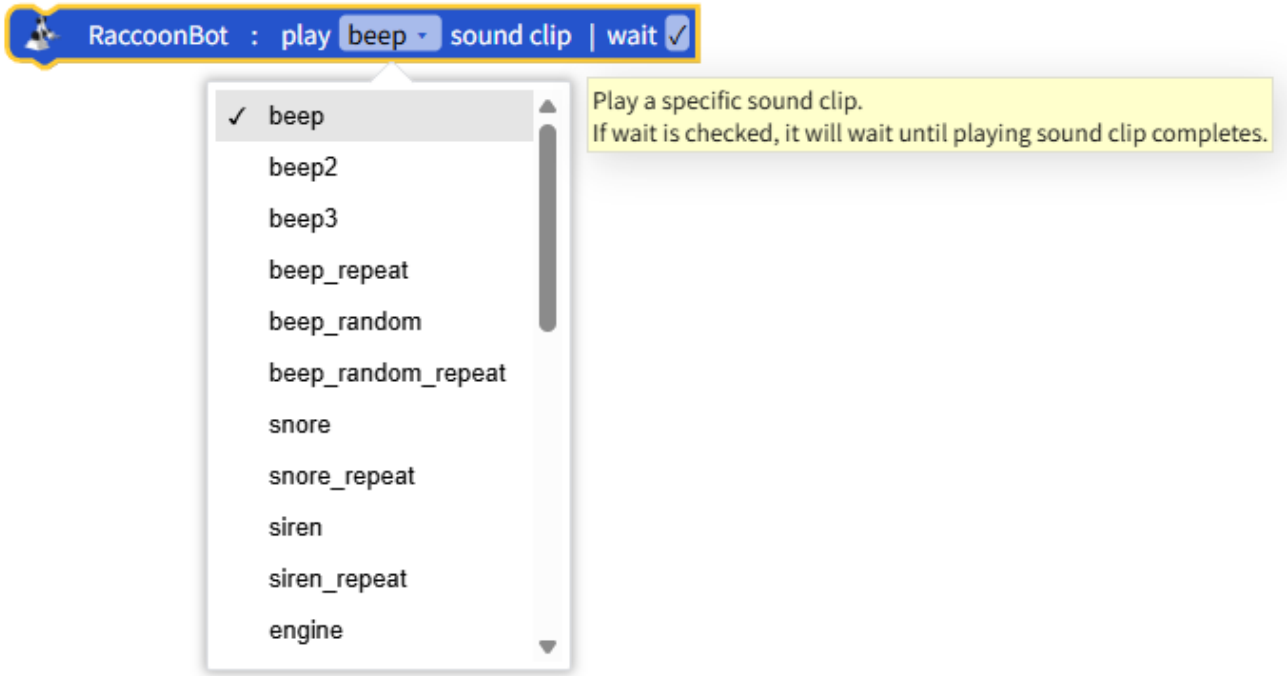
```
# Play Do in octave 1
__('Raccoon4*0:sound.note').d = 4

# Play Re in octave 1
__('Raccoon4*0:sound.note').d = 6

# Play Do in octave 2
__('Raccoon4*0:sound.note').d = 16

# Play Si in octave 7
__('Raccoon4*0:sound.note').d = 87
```

Play a Sound

The RaccoonBot plays a specific sound clip.

If you check Wait, it waits until playback is completed.

However, if Wait is checked, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
sound_clip	Dropdown option	Sound clip	beep(1), beep2(2), beep3(3), beep_repeat(4), beep_random(5), beep_random_repeat(6), snore(7), snore_repeat(8), siren(9), siren_repeat(10), engine(11), engine_repeat(12), fart_a(13), fart_b(14), noise(15), noise_repeat(16), whistle(17), chop_chop(18), chop_chop_repeat(19), random(20), r2d2(21), connected(22), dee_bee(33), finish(34), power_on(35), start(36), power_off(37)

JavaScript Code

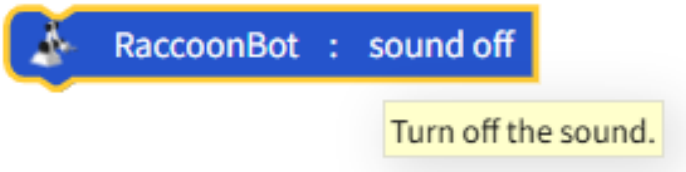
```
// Play finish(34) sound | Wait O
$('Raccoon4*0:sound.clip').d = 34;
await $('Raccoon4*0:sound.!clip').w();

// Play power_on(35) sound | Wait X
$('Raccoon4*0:sound.clip').d = 35;
```

Python Code

```
# Play finish(34) sound | Wait O
__('Raccoon4*0:sound.clip').d = 34
await __('Raccoon4*0:sound.!clip').w()

# Play power_on(35) sound | Wait X
__('Raccoon4*0:sound.clip').d = 35
```

Stop Sound

Turns off the sound of the RaccoonBot.



JavaScript Code

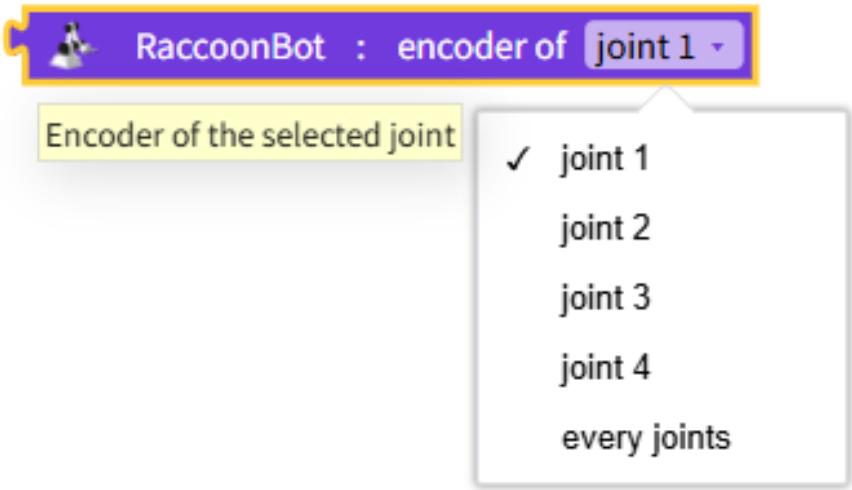
```
// Stop sound
__stopSound('Raccoon4*0');
```

Python Code

```
# Stop sound
__stopSound('Raccoon4*0')
```

Joint Encoder Value

Returns the encoder value of the selected joint.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
motor	Dropdown option	Joint number	joint 1(joint_1), joint 2(joint_2), joint 3(joint_3), joint 4(joint_4), every joints(joint_1, joint_2, joint_3, joint_4)

JavaScript Code

```
// Joint 1 encoder value
$('Raccoon4*0:encoder.joint_1').d;

// Joint 2 encoder value
$('Raccoon4*0:encoder.joint_2').d;

// Joint 3 encoder value
$('Raccoon4*0:encoder.joint_3').d;

// Joint 4 encoder value
```

```
$('Raccoon4*0:encoder.joint_4').d;

// All joint encoder values
[$('Raccoon4*0:encoder.joint_1').d, $('Raccoon4*0:encoder.joint_2').d,
$('Raccoon4*0:encoder.joint_3').d, $('Raccoon4*0:encoder.joint_4').d];
```

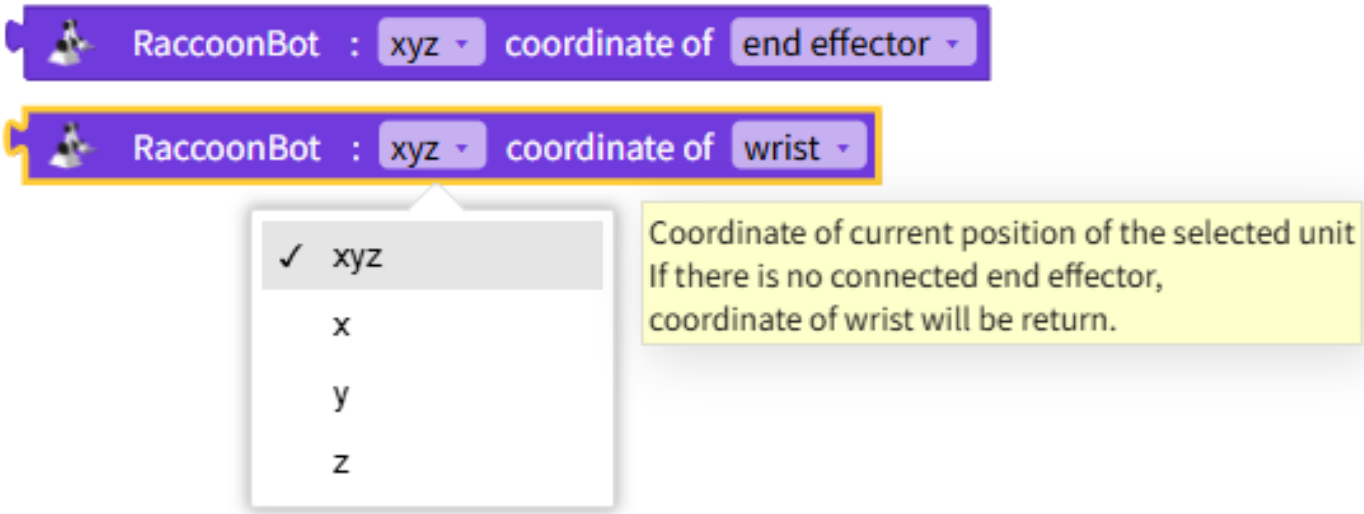
Python Code

```
# Joint 1 encoder value
__('Raccoon4*0:encoder.joint_1').d

# Joint 2 encoder value
__('Raccoon4*0:encoder.joint_2').d

# Joint 3 encoder value
__('Raccoon4*0:encoder.joint_3').d

# Joint 4 encoder value
__('Raccoon4*0:encoder.joint_4').d

# All joint encoder values
[__('Raccoon4*0:encoder.joint_1').d, __('Raccoon4*0:encoder.joint_2').d,
__('Raccoon4*0:encoder.joint_3').d, __('Raccoon4*0:encoder.joint_4').d]
```

Coordinates of the Selected Origin

Returns the current coordinates of the selected origin.

If there is no connected end-effector device, the wrist-based position is returned.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
origin	Dropdown option	selected origin	wrist (wrist), end-effector (end_effector)
coordinate	Dropdown option	Coordinate axis	xyz, x, y, z

JavaScript Code

```
// Wrist-based xyz coordinates
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'wrist');

// Wrist-based x coordinate
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'wrist')[0];

// Wrist-based y coordinate
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'wrist')[1];
```

```
// Wrist-based z coordinate
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'wrist')[2];

// end-effector-based xyz coordinates
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'end_effector');

// end-effector-based x coordinate
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'end_effector')[0];

// end-effector-based y coordinate
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'end_effector')[1];

// end-effector-based z coordinate
__angles_to_xyz('Raccoon4*0', [$('Raccoon4*0:encoder.joint_1').d,
$('Raccoon4*0:encoder.joint_2').d, $('Raccoon4*0:encoder.joint_3').d,
$('Raccoon4*0:encoder.joint_4').d], 'end_effector')[2];
```

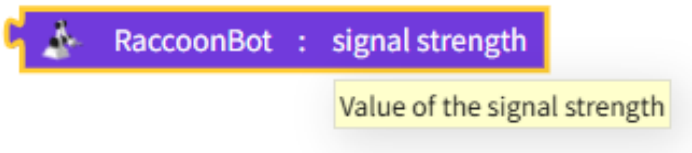
Python Code

```
# Wrist-based xyz coordinates
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'wrist')

# Wrist-based x coordinate
```

```
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'wrist')[0]

# Wrist-based y coordinate
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'wrist')[1]

# Wrist-based z coordinate
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'wrist')[2]

# end-effector-based xyz coordinates
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'end_effector')

# end-effector-based x coordinate
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'end_effector')[0]

# end-effector-based y coordinate
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'end_effector')[1]

# end-effector-based z coordinate
__angles_to_xyz('Raccoon4*0', [__('Raccoon4*0:encoder.joint_1').d,
__('Raccoon4*0:encoder.joint_2').d, __('Raccoon4*0:encoder.joint_3').d,
__('Raccoon4*0:encoder.joint_4').d], 'end_effector')[2]
```

Signal Strength Value

Returns the signal strength of the RaccoonBot.



JavaScript Code

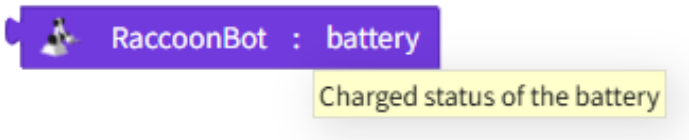
```
// RaccoonBot signal strength
$('Raccoon4*0:signal_strength').d;
```

Python Code

```
# RaccoonBot signal strength
__('Raccoon4*0:signal_strength').d
```

Battery Charge Level Value

Returns the battery charge level value of the RaccoonBot.



JavaScript Code

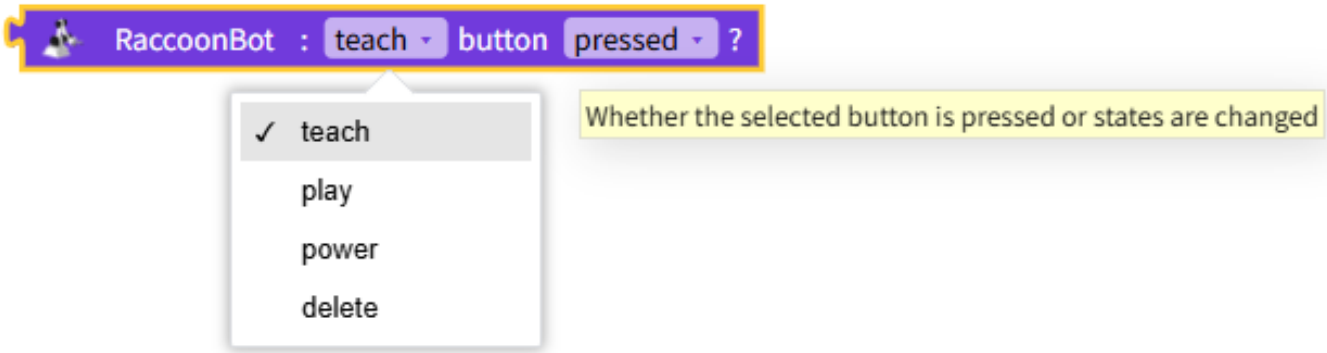
```
// RaccoonBot battery charge level value
$('Raccoon4*0:battery.level').d;
```

Python Code

```
# RaccoonBot battery charge level value
__('Raccoon4*0:battery.level').d
```

Is the Button Pressed?

Returns **true(1) / false(0)** depending on whether the selected button is pressed or a click event occurred. However, when long-clicking the power button, it is Power ON/OFF, so you cannot check whether it was long-clicked.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
button	Dropdown option	Button type	teach, play, power, delete
state	Dropdown option	Click event	pressed, click, long click

JavaScript Code

```
// Is the teach button pressed?
$('Raccoon4*0:button.pressed.teach').d;
// Was the teach button clicked?
$('Raccoon4*0:button.click.teach').e;
// Was the teach button long-clicked?
$('Raccoon4*0:button.long_click.teach').e;

// Is the play button pressed?
$('Raccoon4*0:button.pressed.play').d;
// Was the play button clicked?
$('Raccoon4*0:button.click.play').e;
// Was the play button long-clicked?
$('Raccoon4*0:button.long_click.play').e;
```

```
// Is the power button pressed?
$('Raccoon4*0:button.pressed.power').d;
// Was the power button clicked?
$('Raccoon4*0:button.click.power').e;
// Was the power button long-clicked? -> X

// Is the delete button pressed?
$('Raccoon4*0:button.pressed.delete').d;
// Was the delete button clicked?
$('Raccoon4*0:button.click.delete').e;
// Was the delete button long-clicked?
$('Raccoon4*0:button.long_click.delete').e;
```

Python Code

```
# Is the teach button pressed?
__('Raccoon4*0:button.pressed.teach').d
# Was the teach button clicked?
__('Raccoon4*0:button.click.teach').e
# Was the teach button long-clicked?
__('Raccoon4*0:button.long_click.teach').e

# Is the play button pressed?
__('Raccoon4*0:button.pressed.play').d
# Was the play button clicked?
__('Raccoon4*0:button.click.play').e
# Was the play button long-clicked?
__('Raccoon4*0:button.long_click.play').e

# Is the power button pressed?
__('Raccoon4*0:button.pressed.power').d
# Was the power button clicked?
__('Raccoon4*0:button.click.power').e
```

```
# Was the power button long-clicked? -> X

# Is the delete button pressed?
__('Raccoon4*0:button.pressed.delete').d
# Was the delete button clicked?
__('Raccoon4*0:button.click.delete').e
# Was the delete button long-clicked?
__('Raccoon4*0:button.long_click.delete').e
```

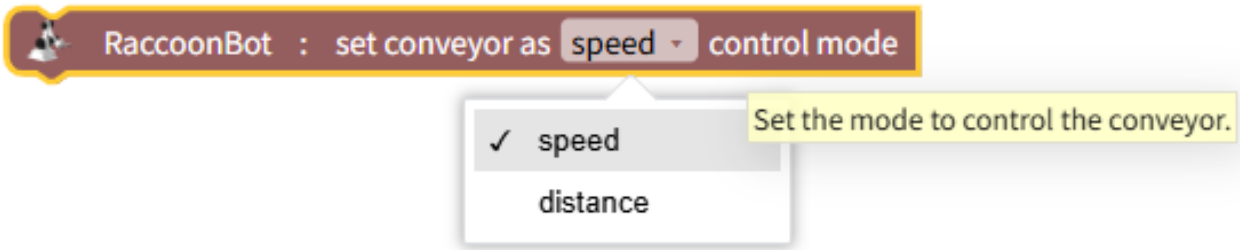
Set Conveyor Mode

Sets the mode for controlling the conveyor.

In speed control mode, the conveyor is moved only by controlling the conveyor speed.

In distance control mode, the conveyor is moved only by controlling the distance.

The two modes cannot be used simultaneously.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
mode	Dropdown option	Conveyor mode	speed (0), distance (1)

JavaScript Code

```
// Set the conveyor to speed control mode
$('Raccoon4*0:peripheral>conveyor+mode').d = 0;

// Set the conveyor to distance control mode
$('Raccoon4*0:peripheral>conveyor+mode').d = 1;
```

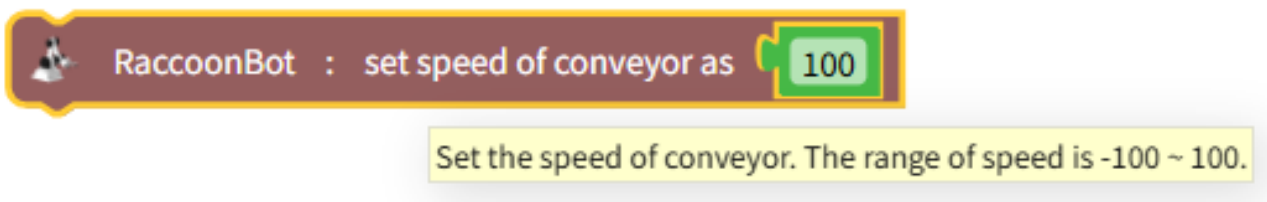
Python Code

```
# Set the conveyor to speed control mode
__('Raccoon4*0:peripheral>conveyor+mode').d = 0

# Set the conveyor to distance control mode
__('Raccoon4*0:peripheral>conveyor+mode').d = 1
```

Set Conveyor Speed

Sets the speed of the conveyor belt.

The speed range is -100 ~ 100.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
speed	Input value	Conveyor speed	Real number -100 ~ 100

JavaScript Code

```
// Set the conveyor speed ~ 100
$('Raccoon4*0:peripheral>conveyor+speed').d = 100;
```

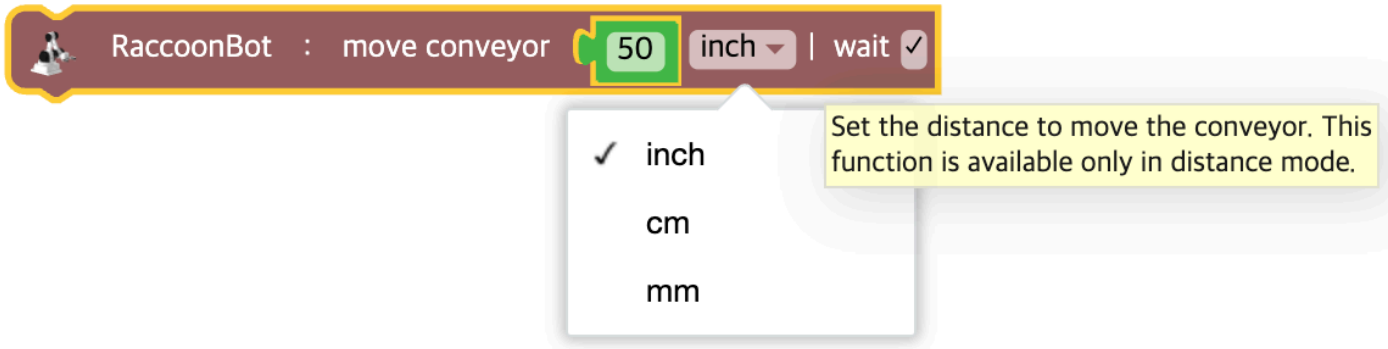
Python Code

```
# Set the conveyor speed ~ 100
__('Raccoon4*0:peripheral>conveyor+speed').d = 100
```

Set Conveyor Move Distance

Sets the distance to move the conveyor belt.

This function can only be used in distance control mode.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
distance	Input value	Distance	Real number >= 0
unit	Dropdown option	Distance unit	cm, mm, inch

JavaScript Code

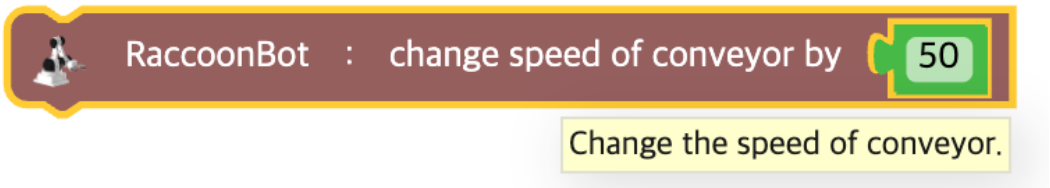
```
// Move the conveyor by 100cm | Wait O
$('Raccoon4*0:peripheral>conveyor+distance').d = __getDistance('Conveyor', 100, 'cm');
await $('Raccoon4*0:peripheral>conveyor+!distance').w();

// Move the conveyor by 100mm | Wait O
$('Raccoon4*0:peripheral>conveyor+distance').d = __getDistance('Conveyor', 100, 'mm');
await $('Raccoon4*0:peripheral>conveyor+!distance').w();

// Move the conveyor by 100inch | Wait X
$('Raccoon4*0:peripheral>conveyor+distance').d = __getDistance('Conveyor', 100, 'inch');
```

Python Code

```
# Move the conveyor by 100cm | Wait O
__('Raccoon4*0:peripheral>conveyor+distance').d = __getDistance('Conveyor', 100,
'cm')
await __('Raccoon4*0:peripheral>conveyor+!distance').w()

# Move the conveyor by 100mm | Wait O
__('Raccoon4*0:peripheral>conveyor+distance').d = __getDistance('Conveyor', 100,
'mm')
await __('Raccoon4*0:peripheral>conveyor+!distance').w()

# Move the conveyor by 100inch | Wait X
__('Raccoon4*0:peripheral>conveyor+distance').d = __getDistance('Conveyor', 100,
'inch')
```

Change Conveyor Speed

Changes the speed of the conveyor belt by the input value.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
speed	Input value	Speed change	Real number -200 ~ 200

JavaScript Code

```
// Change the conveyor speed by -200
$('Raccoon4*0:peripheral>conveyor+speed').d =
$('Raccoon4*0:peripheral>conveyor+speed').d + (-200);
```

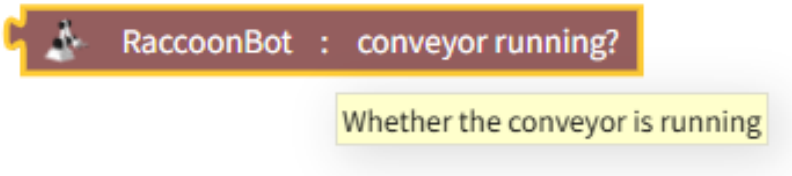
```
// Change the conveyor speed by 200
$('Raccoon4*0:peripheral>conveyor+speed').d =
$('Raccoon4*0:peripheral>conveyor+speed').d + 200;
```

Python Code

```
# Change the conveyor speed by -200
__('Raccoon4*0:peripheral>conveyor+speed').d =
__('Raccoon4*0:peripheral>conveyor+speed').d + (-200)

# Change the conveyor speed by 200
__('Raccoon4*0:peripheral>conveyor+speed').d =
__('Raccoon4*0:peripheral>conveyor+speed').d + 200
```

Is the Conveyor Running?

Returns **true(1) / false(0)** depending on whether the conveyor belt is running.



JavaScript Code

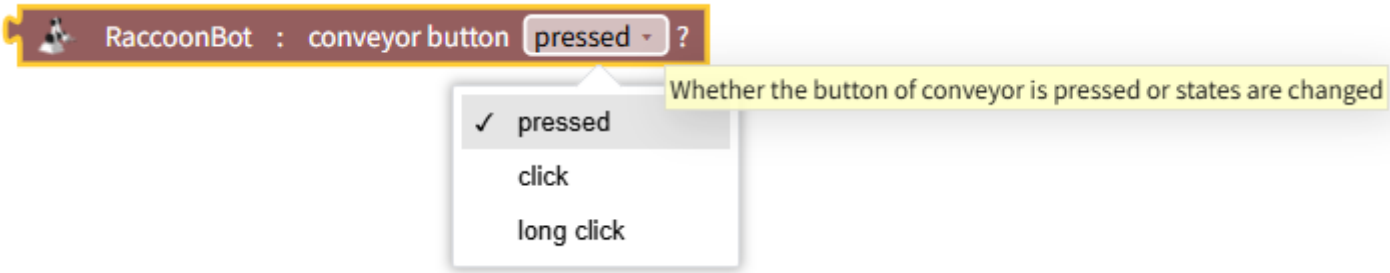
```
// Is the conveyor running?
$('Raccoon4*0:peripheral>conveyor+running').d;
```

Python Code

```
# Is the conveyor running?
__('Raccoon4*0:peripheral>conveyor+running').d
```

Is the Conveyor Button Pressed?

Returns **true(1) / false(0)** depending on whether the conveyor button is pressed or a click event occurred.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
state	Dropdown option	Click event	pressed, click, long click

JavaScript Code

```
// Is the conveyor button pressed?
$('Raccoon4*0:peripheral>conveyor+button.pressed').d;

// Was the conveyor button clicked?
$('Raccoon4*0:peripheral>conveyor+button.click').e;

// Was the conveyor button long-clicked?
$('Raccoon4*0:peripheral>conveyor+button.long_click').e;
```

Python Code

```
# Is the conveyor button pressed?
__('Raccoon4*0:peripheral>conveyor+button.pressed').d

# Was the conveyor button clicked?
__('Raccoon4*0:peripheral>conveyor+button.click').e
```

```
# Was the conveyor button long-clicked?
__('Raccoon4*0:peripheral>conveyor+button.long_click').e
```